

GMADL这个函数有什么问题，画图，想一下改进方法

☒ Done	☐
Due Date	@2025年10月12日

任务：

- 想一下GMADL这个函数有什么问题，比如，true return = -0.1（或者+0.1），estimated return = -1, -0.5, -0.1, -0.01, +0.01, +0.1, +0.5, +1的时候，GMADL有什么变化。这样的变化是不是都是合理的？
- 可以再画个图比较一下。
- 然后思考一下如果有什么问题，可以怎么改进？

任务一：GMADL函数测试矩阵

测试配置

参数设定：a = 100, b = 2（与论文一致）

测试场景：

- True return: **+0.1** 和 **-0.1**
- Estimated return: -1, -0.5, -0.1, -0.01, +0.01, +0.1, +0.5, +1

计算公式回顾

$$L_i = -\left[\sigma(a \cdot y \cdot \hat{y}) - 0.5\right] \times |y|^b$$

其中：

- y ：观测值（true return）
- $\hat{y}$ ：预测值（estimated return）
- $\sigma(x) = \frac{1}{1 + e^{-x}}$ ：sigmoid函数
- $a = 100$ ：控制sigmoid陡度
- $b = 2$ ：控制观测值权重

测试表格A：True Return = +0.1

Est. Return	$y \cdot \hat{y}$	$a \cdot y \cdot \hat{y}$	$\sigma(a \cdot y \cdot \hat{y})$	$\sigma - 0.5$	$ y ^2$	Loss L_i
-1.0	-0.1	-10	0.0000454	-0.4999546	0.01	+0.0049995 × 惩罚
-0.5	-0.05	-5	0.00669	-0.49331	0.01	+0.0049331 × 惩罚
-0.1	-0.01	-1	0.2689	-0.2311	0.01	+0.002311 × 惩罚
-0.01	-0.001	-0.1	0.475	-0.025	0.01	+0.00025 × 惩罚
+0.01	+0.001	+0.1	0.525	+0.025	0.01	-0.00025 ✓ 奖励
+0.1	+0.01	+1	0.7311	+0.2311	0.01	-0.002311 ✓ 奖励

Est. Return	$y \cdot \hat{y}$	$a \cdot y \cdot \hat{y}$	$\sigma(a \cdot y \cdot \hat{y})$	$\sigma - 0.5$	$ y ^2$	Loss L_i
+0.5	+0.05	+5	0.9933	+0.4933	0.01	-0.004933 ✓ 奖励
+1.0	+0.1	+10	0.999955	+0.499955	0.01	-0.00499955 ✓奖励

测试表格B：True Return = -0.1

Est. Return	$y \cdot \hat{y}$	$a \cdot y \cdot \hat{y}$	$\sigma(a \cdot y \cdot \hat{y})$	$\sigma - 0.5$	$ y ^2$	Loss L_i
-1.0	+0.1	+10	0.999955	+0.499955	0.01	-0.00499955 ✓奖励
-0.5	+0.05	+5	0.9933	+0.4933	0.01	-0.004933 ✓ 奖励
-0.1	+0.01	+1	0.7311	+0.2311	0.01	-0.002311 ✓奖励
-0.01	+0.001	+0.1	0.525	+0.025	0.01	-0.00025 ✓奖励
+0.01	-0.001	-0.1	0.475	-0.025	0.01	+0.00025 ×惩罚
+0.1	-0.01	-1	0.2689	-0.2311	0.01	+0.002311 ×惩罚
+0.5	-0.05	-5	0.00669	-0.49331	0.01	+0.0049331 ×惩罚
+1.0	-0.1	-10	0.0000454	-0.4999546	0.01	+0.0049995 ×惩罚

关键观察维度

1. 方向判断有效性

- ☐ sigmoid在 ± 0.01 附近是否能准确区分方向？
- ☐ 临界点附近 ($\hat{y}$ 接近0) 是否存在判断模糊区？

2. 奖惩幅度对称性

- ☐ 方向正确的奖励与方向错误的惩罚是否对称？
- ☐ 这种对称性对交易策略是否合理？

3. 极端预测处理

- ☐ 当 $\hat{y}=\pm 1$ （极端预测）但 $y=\pm 0.1$ （中等观测）时，奖惩是否过度？
- ☐ 是否应引入对预测值的约束？

4. 梯度连续性

- ☐ Loss值从负到正的过渡是否平滑？
- ☐ 是否存在梯度消失或爆炸风险？

预期发现问题

潜在问题1：小预测值判断失效

当 $|\hat{y}| < 0.01$ 时，sigmoid项接近0.5，方向判断信号微弱

潜在问题2：对称性过强

奖励幅度 = 惩罚幅度，但交易中"避免亏损 > 追求盈利"

潜在问题3：缺乏预测置信度调节

所有预测值同等对待，未考虑模型本身的不确定性

任务二：可视化方案代码

方案B：多子图线图（优先）

目标：深入理解GMADL在不同True Return场景下的连续行为

```
import numpy as np
import matplotlib.pyplot as plt

# GMADL函数定义
def gmadl(y, y_hat, a=100, b=2):
    """
    计算GMADL损失
    y: 观测值 (true return)
    y_hat: 预测值 (estimated return)
    a: sigmoid陡度参数
    b: 观测值权重参数
    """
    product = a * y * y_hat
    sigmoid = 1 / (1 + np.exp(-product))
    loss = -(sigmoid - 0.5) * np.abs(y) ** b
    return loss

# 设置绘图参数
plt.rcParams['font.sans-serif'] = ['Arial Unicode MS', 'SimHei'] # 支持中文
plt.rcParams['axes.unicode_minus'] = False

# 生成预测值范围（连续）
y_hat_range = np.linspace(-1, 1, 500)

# 选择4个代表性的True Return值
true_returns = [-0.8, -0.1, 0.1, 0.8]

# 创建2x2子图
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for idx, y_true in enumerate(true_returns):
    ax = axes[idx]

    # 计算该True Return下所有预测值的Loss
    losses = [gmadl(y_true, y_hat) for y_hat in y_hat_range]

    # 绘制Loss曲线
    ax.plot(y_hat_range, losses, linewidth=2.5, color='#2E86AB', label='GMADL')

    # 标注方向切换点（x=0）
    ax.axvline(x=0, color='red', linestyle='--', linewidth=1.5, alpha=0.7, label='方向切换点')
```

```

# 标注奖励/惩罚区域
ax.axhline(y=0, color='gray', linestyle='-', linewidth=0.8, alpha=0.5)
ax.fill_between(y_hat_range, 0, losses, where=(np.array(losses) < 0),
                color='green', alpha=0.2, label='奖励区')
ax.fill_between(y_hat_range, 0, losses, where=(np.array(losses) > 0),
                color='red', alpha=0.2, label='惩罚区')

# 设置标题和标签
ax.set_title(f'True Return = {y_true:+.1f}', fontsize=13, fontweight='bold')
ax.set_xlabel('Estimated Return', fontsize=11)
ax.set_ylabel('GMADL Loss', fontsize=11)
ax.grid(True, alpha=0.3)
ax.legend(fontsize=9, loc='best')

# 标注关键点
if idx == 0: # 只在第一个子图显示完整图例
    ax.text(0.05, 0.95, f'方向正确区:  $\hat{y} < 0$ \n方向错误区:  $\hat{y} > 0$ ',
            transform=ax.transAxes, fontsize=9, verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.savefig('gmadl_multiplot.png', dpi=300, bbox_inches='tight')
plt.show()

print("✓ 方案B完成：多子图线图已保存为 gmadl_multiplot.png")

```

方案A：热力图（全局视角）

目标： 一图呈现GMADL在全参数空间的行为模式

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# GMADL函数定义
def gmadl(y, y_hat, a=100, b=2):
    product = a * y * y_hat
    sigmoid = 1 / (1 + np.exp(-product))
    loss = -(sigmoid - 0.5) * np.abs(y) ** b
    return loss

# 生成网格数据
y_range = np.linspace(-1, 1, 200)
y_hat_range = np.linspace(-1, 1, 200)
Y, Y_hat = np.meshgrid(y_range, y_hat_range)

# 计算每个网格点的Loss
Z = np.zeros_like(Y)
for i in range(Y.shape[0]):
    for j in range(Y.shape[1]):
        Z[i, j] = gmadl(Y[i, j], Y_hat[i, j])

```

```

# 绘制热力图
fig, ax = plt.subplots(figsize=(12, 10))

# 使用diverging colormap（红绿对比）
im = ax.contourf(Y, Y_hat, Z, levels=50, cmap='RdYlGn_r', vmin=-0.5, vmax=0.5)

# 添加对角线（方向正确的边界）
ax.plot([-1, 1], [-1, 1], 'k--', linewidth=2, label='方向边界（ $y = \hat{y}$ ）')
ax.plot([-1, 1], [1, -1], 'k--', linewidth=2)

# 添加颜色条
cbar = plt.colorbar(im, ax=ax)
cbar.set_label('GMADL Loss（负=奖励，正=惩罚）', fontsize=12)

# 标注区域
ax.text(0.5, 0.5, '方向正确\n（奖励区）', fontsize=14, ha='center',
       bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.7))
ax.text(-0.5, 0.5, '方向错误\n（惩罚区）', fontsize=14, ha='center',
       bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.7))
ax.text(0.5, -0.5, '方向错误\n（惩罚区）', fontsize=14, ha='center',
       bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.7))
ax.text(-0.5, -0.5, '方向正确\n（奖励区）', fontsize=14, ha='center',
       bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.7))

# 设置标签
ax.set_xlabel('True Return (y)', fontsize=13, fontweight='bold')
ax.set_ylabel('Estimated Return ( $\hat{y}$ )', fontsize=13, fontweight='bold')
ax.set_title('GMADL损失函数全景热力图 (a=100, b=2)', fontsize=15, fontweight='bold', pad=20)
ax.grid(True, alpha=0.3, linestyle=':')
ax.legend(fontsize=11)

plt.tight_layout()
plt.savefig('gmadl_heatmap.png', dpi=300, bbox_inches='tight')
plt.show()

print("✓ 方案A完成：热力图已保存为 gmadl_heatmap.png")

```

方案C：MADL vs GMADL对比（论文级）

目标： 突出两种损失函数的差异，用于论文实验对比

```

import numpy as np
import matplotlib.pyplot as plt

# MADL函数定义
def madl(y, y_hat):
    """
    计算MADL损失
    """
    if y * y_hat > 0: # 方向正确
        return -np.abs(y)
    elif y * y_hat < 0: # 方向错误
        return np.abs(y)

```

```

else: # 预测为0
    return 0

# GMADL函数定义
def gmadl(y, y_hat, a=100, b=2):
    product = a * y * y_hat
    sigmoid = 1 / (1 + np.exp(-product))
    loss = -(sigmoid - 0.5) * np.abs(y) ** b
    return loss

# 生成预测值范围
y_hat_range = np.linspace(-1, 1, 500)

# 选择2个关键场景（沿用论文示例）
scenarios = [
    {'y': 0.01, 'title': 'True Return = +0.01 (小幅上涨)'},
    {'y': -0.8, 'title': 'True Return = -0.8 (大幅下跌)'}
]

# 创建1x2对比图
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for idx, scenario in enumerate(scenarios):
    ax = axes[idx]
    y_true = scenario['y']

    # 计算MADL和GMADL
    madl_losses = [madl(y_true, y_hat) for y_hat in y_hat_range]
    gmadl_losses = [gmadl(y_true, y_hat) for y_hat in y_hat_range]

    # 绘制两条曲线
    ax.plot(y_hat_range, madl_losses, linewidth=2.5, color='#A23B72',
            label='MADL', linestyle='--', marker='o', markevery=50, markersize=4)
    ax.plot(y_hat_range, gmadl_losses, linewidth=2.5, color='#2E86AB',
            label='GMADL', marker='s', markevery=50, markersize=4)

    # 标注方向切换点
    ax.axvline(x=0, color='red', linestyle=':', linewidth=1.5, alpha=0.7, label='方向切换点')
    ax.axhline(y=0, color='gray', linestyle='-', linewidth=0.8, alpha=0.5)

    # 设置标题和标签
    ax.set_title(scenario['title'], fontsize=14, fontweight='bold')
    ax.set_xlabel('Estimated Return', fontsize=12)
    ax.set_ylabel('Loss Value', fontsize=12)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=11, loc='best')

    # 标注关键差异点
    if idx == 0: # 小幅上涨场景
        ax.text(0.5, -0.005, 'GMADL奖励更平滑', fontsize=10, ha='center',
                bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.6))
    else: # 大幅下跌场景
        ax.text(0.5, 0.4, 'GMADL惩罚更严厉\n（约2倍MADL）', fontsize=10, ha='center',
                bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.6))

```

```
plt.suptitle('MADL vs GMADL 对比实验 (a=100, b=2) ', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('madl_vs_gmadl_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

print("✓ 方案C完成：MADL vs GMADL对比图已保存为 madl_vs_gmadl_comparison.png")
```

使用说明

运行顺序：

1. **先运行方案B** → 观察GMADL在不同场景的斜率、连续性、对称性
2. **再运行方案A** → 从全局视角验证方向判断的边界清晰度
3. **最后运行方案C** → 量化GMADL相对MADL的改进幅度

关键观察点：

- 方案B：关注临界点（ $x=0$ 附近）的平滑度，是否存在梯度消失
- 方案A：检查对角线两侧颜色过渡是否清晰，是否有模糊地带
- 方案C：对比大波动场景（ $y=-0.8$ ）时GMADL的惩罚是否显著强于MADL